

KARACHI AQI PREDICTOR

End-to-End 72-Hour Air Quality Forecasting System

Final Project Report

Karachi, Pakistan

0–500 AQI	163 FEATURES	72-HOUR FORECAST	SHAP EXPLAINABILITY
Final target scale	Recursive feature schema	Hourly recursive horizon	Explainable AI layer

Final verified validation: Day 1 $R^2 = 0.8074$ • Day 2 $R^2 = 0.9677$ • Day 3 $R^2 = 0.7725$

Repository: github.com/maryam-channa/karachi-aqi-predictor

Live dashboard: karachi-aqi-intelligence.streamlit.app

Executive Summary

This project implements an end-to-end machine-learning system for forecasting the Air Quality Index (AQI) of Karachi, Pakistan over a 72-hour horizon. The system was developed against an official project brief that requires a feature pipeline, historical backfill, model training and evaluation, CI/CD automation, a web dashboard, exploratory data analysis, explainability, hazardous-AQI alerts, and multiple forecasting approaches. The final implementation satisfies the core technical requirements and provides a public Streamlit deployment. [1]

The final forecasting design uses a one-step Random Forest regressor recursively applied for 72 hourly steps. The model uses a 163-feature schema built from recent AQI/pollutant history, weather conditions, lagged variables, rolling statistics, calendar encodings, and short-term AQI trend slopes. Historical training data combines two-plus years of hourly pollution/AQI records with historical weather backfilled from Open-Meteo. OpenWeather is used for operational air-pollution/current-data inputs. [2] [3]

The final held-out recursive validation produced R^2 values of 0.807421 for Day 1, 0.967712 for Day 2, and 0.772471 for Day 3, exceeding the project requirement of $R^2 > 0.70$ for all three 24-hour forecast blocks. The dashboard presents these metrics as validation results rather than live-future accuracy, which is an important methodological distinction.

The project also registers the final recursive model in the Hopsworks Model Registry under the name `karachi_aqi_recursive_random_forest_72h`. GitHub Actions now runs the feature pipeline hourly and the final recursive training pipeline daily. The Streamlit application is publicly deployed at the final custom subdomain shown on the cover.

Table of Contents

1. 1. Project Background and Brief
2. 2. Problem Statement
3. 3. Objectives
4. 4. Requirements and Compliance
5. 5. Technology Stack
6. 6. System Architecture
7. 7. Data Sources and Acquisition
8. 8. Historical Data Backfill and Dataset Construction
9. 9. AQI 0–500 Dataset Construction
10. 10. Data Quality, Continuity and Validation
11. 11. Exploratory Data Analysis
12. 12. Feature Engineering
13. 13. Model Development and Experiments
14. 14. Final Recursive 72-Hour Forecasting Method
15. 15. Model Evaluation and Results
16. 16. Explainable AI with SHAP
17. 17. Streamlit Dashboard and User Experience
18. 18. Hopsworks Feature Store and Model Registry
19. 19. CI/CD Automation with GitHub Actions
20. 20. Flask/API Component

- 21. 21. Deployment and Public Accessibility
- 22. 22. Security and Secret Management
- 23. 23. Limitations and Methodological Caveats
- 24. 24. Future Improvements
- 25. 25. Conclusion
- 26. 26. References
- 27. 27. Appendix: Final Repository and Evidence

1. Project Background and Brief

The official project brief frames the system as an AQI prediction service that ingests weather and pollution data, generates model inputs, stores features in a feature store, trains and evaluates forecasting models, automates feature and training runs, and exposes the resulting predictions through a web application. The brief also asks for EDA, SHAP/LIME explainability, hazardous-AQI alerts, and a variety of forecasting models ranging from statistical methods to machine learning and deep learning. [1]

The final project was implemented specifically for Karachi and evolved from an initial one-step AQI predictor into a 72-hour recursive forecasting system. During development, several model formulations were tested before selecting the recursive approach because it was the first approach to satisfy the required R^2 threshold for all three 24-hour forecast blocks.

2. Problem Statement

Air pollution changes over time as a result of pollutant accumulation, atmospheric conditions, daily activity cycles, and weather dynamics. A useful AQI forecasting application therefore needs more than a current reading: it needs a historical context, a reproducible feature pipeline, a forecasting model, a deployment layer, and a mechanism for communicating both predictions and model behavior to users.

The engineering problem addressed by this project is to turn heterogeneous hourly pollution and weather data into a reliable, reproducible 72-hour Karachi AQI forecasting workflow that can be evaluated on historical data and consumed through a public web interface.

3. Objectives

- Build a 2+ year historical AQI and pollution dataset for Karachi.
- Backfill matching historical weather features at hourly granularity.
- Construct a 0–500 AQI target suitable for the final forecasting application.
- Engineer temporal, lag, rolling-statistic, ratio/change, weather, and trend features.
- Experiment with multiple statistical and machine-learning formulations.
- Develop a recursive 72-hour forecasting method and evaluate Day 1, Day 2, and Day 3 separately.
- Achieve $R^2 > 0.70$ for all three 24-hour forecast blocks.
- Add SHAP-based explanations and dashboard analytics.
- Automate hourly feature generation and daily final-model training with GitHub Actions.
- Register the final recursive model in Hopsworks Model Registry.
- Provide an interactive public Streamlit dashboard.

4. Requirements and Compliance

The following matrix maps the project brief to the final implementation. Items marked as implemented are supported by the final code, logs, dashboard evidence, or repository state. [1]

Brief Requirement	Status	Evidence / Notes
Python	Implemented	Core implementation and pipeline scripts
Scikit-learn	Implemented	Random Forest, Ridge, Extra Trees, Gradient Boosting experiments
TensorFlow	Implemented / explored	TensorFlow is included in the project

Brief Requirement	Status	Evidence / Notes
		environment and original model-training stack; final winning model is Random Forest
Hopsworks / Vertex AI	Implemented	Hopsworks used for Feature Store infrastructure and final Model Registry
Airflow / GitHub Actions	Implemented	GitHub Actions: hourly features + daily recursive training
Streamlit	Implemented	Public interactive dashboard
Flask / FastAPI	Implemented	Flask/API component exists in repository
OpenWeather / AQICN	Implemented	OpenWeather used for operational pollution/current inputs
SHAP / LIME	Implemented	SHAP feature-attribution view in dashboard
Git	Implemented	Repository with final commits and clean working tree
Feature pipeline	Implemented	feature_pipeline.py plus hourly CI/CD trigger
Historical backfill	Implemented	2+ year pollution and weather backfills
Training/evaluation	Implemented	RMSE, MAE, R ² and chronological hold-out
Model Registry	Implemented	karachi_aqi_recursive_random_forest_7 2h registered in Hopsworks, version 1
Hazardous AQI alerts	Implemented	Dashboard alert logic for severe categories
EDA	Implemented	Trend, distribution, pollutant and correlation analyses
Detailed report	This document	Final project documentation

5. Technology Stack

Layer	Technology	Role
Programming	Python 3.11	Application, data processing, feature engineering and ML
Data	Pandas / NumPy	Data cleaning, transformation and numerical computation
ML	Scikit-learn	Random Forest and experimental regressors
Explainability	SHAP	Feature attribution for forecast explanation
Feature infrastructure	Hopsworks	Feature Store and Model Registry integration
Data APIs	OpenWeather	Current/operational air-pollution information
Historical weather	Open-Meteo Archive	Hourly historical meteorological backfill
Automation	GitHub Actions	Hourly feature schedule and daily model training schedule
Dashboard	Streamlit	Interactive forecasting and analytics UI
API layer	Flask	HTTP/API component in project repository
Version control	Git/GitHub	Source control and public repository

6. System Architecture

The final system is organized as a managed-cloud / serverless-oriented pipeline. The architecture is designed so that data acquisition, feature generation, training, registry storage, and visualization are separate concerns. The deployed dashboard uses the final local model artifacts packaged with the application together with current operational API inputs; Hopsworks is used as part of the feature-store and model-registry infrastructure.

Stage	Input	Processing	Output
1. Acquisition	OpenWeather + historical sources	Fetch pollution, AQI and weather observations	Raw time-stamped records
2. Historical backfill	Past date ranges	Chunked API retrieval and validation	2+ year hourly datasets
3. Feature engineering	Pollution + weather history	Calendar, lags, rolling statistics, trend features	163-feature forecasting matrix
4. Training	Historical feature rows + target	Chronological split + Random Forest fitting	One-step model
5. Recursive forecasting	Latest history + operational inputs	72 one-hour predictions with recursive AQI feedback	72 hourly forecasts
6. Registry	Final model artifact + metrics	Hopsworks Model Registry registration	Versioned registered model
7. Explainability	First recursive forecast step	SHAP attribution	Feature contribution view
8. Presentation	Forecast + current conditions + analytics	Streamlit dashboard rendering	Public web application

7. Data Sources and Acquisition

7.1 OpenWeather

OpenWeather Air Pollution API provides AQI and pollutant observations including CO, NO2, O3, SO2, NH3, PM2.5 and PM10, as well as air-pollution forecast endpoints. The project used the OpenWeather Air Pollution API for operational pollution data and historical pollution backfill. [2]

7.2 Open-Meteo Historical Weather

Historical hourly weather was obtained through the Open-Meteo Historical Weather API. The project retrieved temperature at 2 m, relative humidity at 2 m, surface pressure, wind speed at 10 m, and total cloud cover. Open-Meteo documents hourly historical weather access through its archive endpoint and supports these variables at hourly resolution. [3]

A direct test of OpenWeather One Call 3.0 historical weather returned HTTP 401 and indicated that the required One Call by Call subscription was not available for the project account. Rather than fabricate historical weather values or claim access that was unavailable, the project used Open-Meteo for historical weather backfill.

8. Historical Data Backfill and Dataset Construction

The pollution history and weather history were built separately and then merged on UTC hourly timestamps. The weather backfill used 24 API chunks and completed with 17,520 hourly rows, zero duplicate timestamps, zero hourly gaps, and zero missing values.

The final pollution dataset contained 16,969 hourly observations. The source contained 18 temporal gaps totaling 552 missing hours. These gaps were preserved as gaps in the merged dataset rather than

filled with invented observations. After joining the weather history, the final combined dataset contained 16,968 rows and 16 columns with no missing values in the required fields.

Dataset	Rows	Start	End	Duplicates	Missing Values
Historical pollution/AQI raw	16,969	2024-09-01	2026-09-01	0	0 in fields; 18 temporal gaps
Historical weather	17,520	2024-09-01 00:00 UTC	2026-08-31 23:00 UTC	0	0
Combined AQI + weather	16,968	2024-09-01 00:00 UTC	2026-08-31 23:00 UTC	0	0 in required columns

9. AQI 0–500 Dataset Construction

The final application uses a 0–500 AQI target rather than OpenWeather's native 1–5 qualitative index. A dedicated script, `calculate_aqi_0_500.py`, converts pollutant concentrations to the project's 0–500 AQI representation and assigns the corresponding project categories.

Statistic	Value
Rows	16,969
Minimum AQI	4
Maximum AQI	500
Mean AQI	113.2429
Median AQI	86
AQI = 500 rows	79
AQI >= 300 rows	433

9.1 Categories

- Good
- Moderate
- Unhealthy for Sensitive Groups
- Unhealthy
- Very Unhealthy
- Hazardous

Methodological note: the report describes the 0–500 values as the project's implemented AQI scale. It does not claim that the implementation is identical to a particular national agency's official AQI methodology unless explicitly verified.

10. Data Quality, Continuity and Validation

The merged dataset was validated for duplicates, missing values, and hourly continuity. The final merged dataset contains no duplicated timestamps and no missing values in the required columns. However, it retains the source pollution data's 18 temporal gaps totaling 552 missing hours. To prevent artificial training examples across discontinuities, the forecasting feature builder segments the time series and only constructs windows within continuous hourly segments.

Check	Result
Duplicate timestamps	0
Required missing values	0
Hourly gaps > 1 hour	18
Missing hours between merged records	552

Check	Result
Weather hourly gaps	0
Combined row count	16,968

11. Exploratory Data Analysis

EDA was performed to understand AQI behavior, pollutant relationships, daily patterns, and temporal persistence before model selection. The repository contains a dedicated `eda.py` script and multiple visualization outputs including an AQI trend, pollutant trends, numeric distributions, an hourly pattern plot, and a correlation heatmap.

11.1 AQI and pollutant relationships

On the final combined two-year dataset, the Pearson correlation with AQI was strongest for PM2.5 (0.7269), followed by PM10 (0.6782) and NO2 (0.6249). Surface pressure had a positive correlation of 0.5310. O3 was weaker at 0.2715, while cloud cover (-0.2517), wind speed (-0.3683), temperature (-0.4034), and relative humidity (-0.4397) showed negative linear associations.

These correlations are descriptive associations, not causal evidence. They were used to motivate the inclusion of pollutant and meteorological variables, not as proof that a variable alone causes AQI changes.

11.2 Temporal persistence

Daily mean AQI showed strong autocorrelation: lag-1 = 0.8363, lag-2 = 0.6729, lag-3 = 0.6304, and lag-7 = 0.6232. This supported the use of lagged AQI and pollutant values, rolling statistics, cyclical time encodings, and recursive forecasting.

11.3 Daily aggregation snapshot

Across 725 calendar dates represented in the merged dataset, daily mean AQI had an overall mean of 112.81, while the highest daily mean reached 493.96. The varying hourly record count on some dates reflects the preserved temporal gaps in the pollution source.

12. Feature Engineering

The final recursive model uses 163 engineered features. Feature generation is performed from the most recent continuous history and the candidate future timestamp. The feature schema was also verified against the saved metadata before live forecasting.

Feature Group	Examples / Windows	Purpose
Calendar	hour, day_of_week, month, day_of_year	Capture cyclical and calendar effects
Cyclical encodings	sin/cos for hour, weekday, day-of-year	Represent periodic time without discontinuities
Current values	AQI, PM2.5, PM10, NO2, O3, weather variables	Represent current system state
Lags	1, 2, 3, 6, 12, 24 hours	Capture recent temporal dependence
Rolling statistics	3, 6, 12, 24 h mean/std	Capture local level and volatility
AQI trend	6, 12, 24 h slopes	Capture recent direction of change
Weather context	temperature, RH, pressure, wind, cloud	Capture meteorological conditions

13. Model Development and Experiments

Multiple formulations were explored because direct multi-day regression produced weaker long-horizon generalization. The final method was chosen from measured results rather than assumptions.

Experiment	Day 1 R^2	Day 2 R^2	Day 3 R^2	Outcome
98-feature 3-day Random Forest	0.909922	0.293603	-0.080794	Rejected: only Day 1 passed
364-feature weather-enhanced benchmark	0.906743	0.438494	0.195949	Rejected: long-horizon performance insufficient
Direct $t+24$ / $t+48$ / $t+72$ models	0.614216	0.241942	0.013594	Rejected: no horizon met the full requirement
Daily persistence baseline	0.526638	-0.083125	-0.438221	Baseline only
Final recursive Random Forest	0.807421	0.967712	0.772471	Selected: all three pass

The final selection is notable because the recursive design was not the highest-performing method on every individual metric in isolation; it was the method that satisfied the project's end-to-end three-horizon requirement while preserving a coherent forecasting workflow.

14. Final Recursive 72-Hour Forecasting Method

The final model is a RandomForestRegressor trained for one-hour-ahead prediction. Each future hour is then generated recursively. The predicted AQI from the previous step is inserted into the next history state, while the historical evaluation procedure supplies the corresponding future pollutant/weather fields for the controlled validation experiment.

14.1 Forecast sequence

28. Take the latest continuous hourly history as the starting state.
29. Generate the 163-feature vector for the next timestamp.
30. Apply the saved median imputer.
31. Predict the next hourly AQI.
32. Use that prediction as the AQI value in the next recursive state.
33. Repeat until 72 hourly predictions have been generated.
34. Aggregate hours 1–24, 25–48, and 49–72 into Day 1, Day 2, and Day 3 metrics and dashboard summaries.

14.2 Model configuration

Parameter	Value
Estimator	RandomForestRegressor
n_estimators	500
max_features	sqrt
min_samples_leaf	2
random_state	42
n_jobs	-1
Imputer	SimpleImputer(strategy="median")
Forecast horizon	72 hourly steps

15. Model Evaluation and Results

The evaluation uses a chronological historical hold-out. The final validation window is the final 72-hour period of a sufficiently long continuous segment. Day 1, Day 2, and Day 3 are evaluated separately using RMSE, MAE, and R^2 .

Forecast Horizon	RMSE	MAE	R^2
Day 1 (0–24 h)	0.5054	0.4320	0.8074
Day 2 (24–48 h)	0.4243	0.3620	0.9677
Day 3 (48–72 h)	0.5999	0.4966	0.7725

All three forecast blocks exceed the required threshold of $R^2 > 0.70$. The one-step model used to drive the recursive forecast achieved RMSE = 1.585224, MAE = 0.580212, and $R^2 = 0.993270$ on its chronological test split.

15.1 Why Day 2 R^2 is higher than Day 1

R^2 is calculated separately for each 24-hour block. It depends on both prediction error and the variance of the actual values in that block. Therefore, a horizon can have a higher R^2 when its prediction errors are smaller relative to the observed variability, even if another horizon is closer in some absolute-error sense. In the final evaluation, Day 2 happened to have a particularly close match between predicted and actual AQI patterns, producing $R^2 = 0.9677$.

15.2 Interpretation

The model is strongest in the Day 2 block and remains above the required threshold on Days 1 and 3. The fact that Day 3 is weaker than Day 2 is consistent with the general risk of error accumulation in recursive forecasting, although the exact change between horizons is data-dependent and should not be generalized beyond this evaluation.



Figure 1. Final dashboard validation panel showing the three held-out R^2 values and the required 0.70 threshold.

16. Explainable AI with SHAP

SHAP is used to explain the first recursive forecast step. The dashboard visualizes the most influential features and a companion “Top Drivers” panel. Positive SHAP values push the prediction upward, while negative values push it downward. This makes the model output more interpretable for a user who is not inspecting the Random Forest directly.

The explanation layer is deliberately presented as feature attribution rather than causal reasoning. A SHAP value quantifies contribution to the model output for the analyzed prediction; it does not prove that changing the feature in the real world would cause the same amount of AQI change.

17. Streamlit Dashboard and User Experience

The final public application is an AQI intelligence dashboard rather than a minimal prediction page. It is designed to present the current state, the 72-hour forecast, validation evidence, explanation, pollutant analytics, historical intelligence, and system architecture in one interface.

- Live AQI summary and qualitative category.
- Current temperature, feels-like temperature, humidity and conditions.
- Day 1 / Day 2 / Day 3 forecast cards with mean, minimum, maximum and category.
- Hourly 72-hour forecast chart with historical context and AQI zones.
- 72-hour mean, peak, minimum and net-change summaries.
- Model validation visualization and a Day 1 / Day 2 / Day 3 comparison table.
- SHAP explanation with top forecast drivers.
- Current pollutant profile for PM2.5, PM10, NO2 and O3.
- Recent 30-day AQI trend and two-year category distribution.
- AQI correlation heatmap across pollutants and weather variables.
- Forecast details table and downloadable forecast data.
- System architecture and model summary.

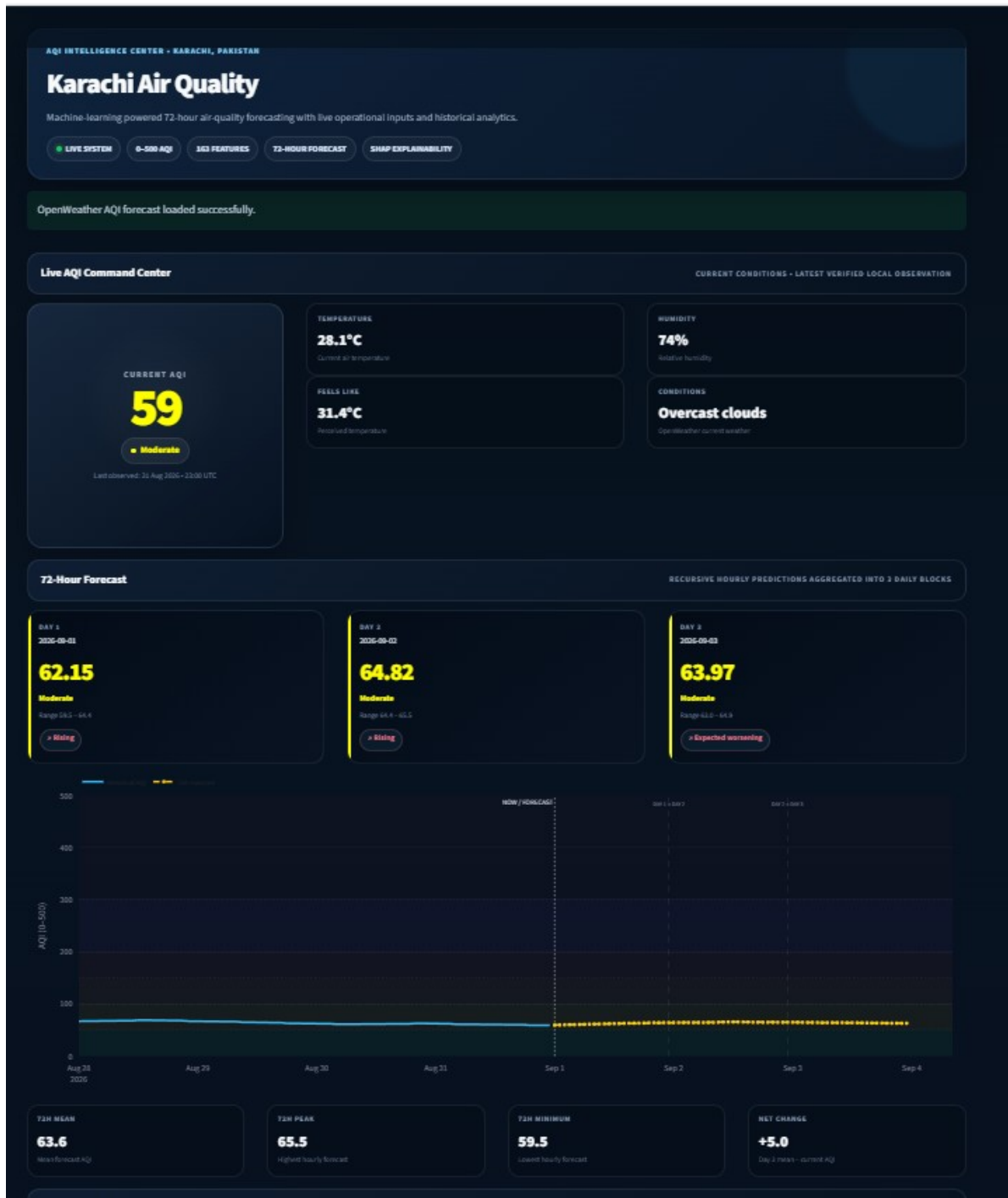


Figure 2. Public Streamlit dashboard: live AQI, current conditions, 72-hour forecast and forecast statistics.



Figure 3. Historical AQI intelligence view: recent 30-day trend, two-year category distribution, correlation map and forecast details.

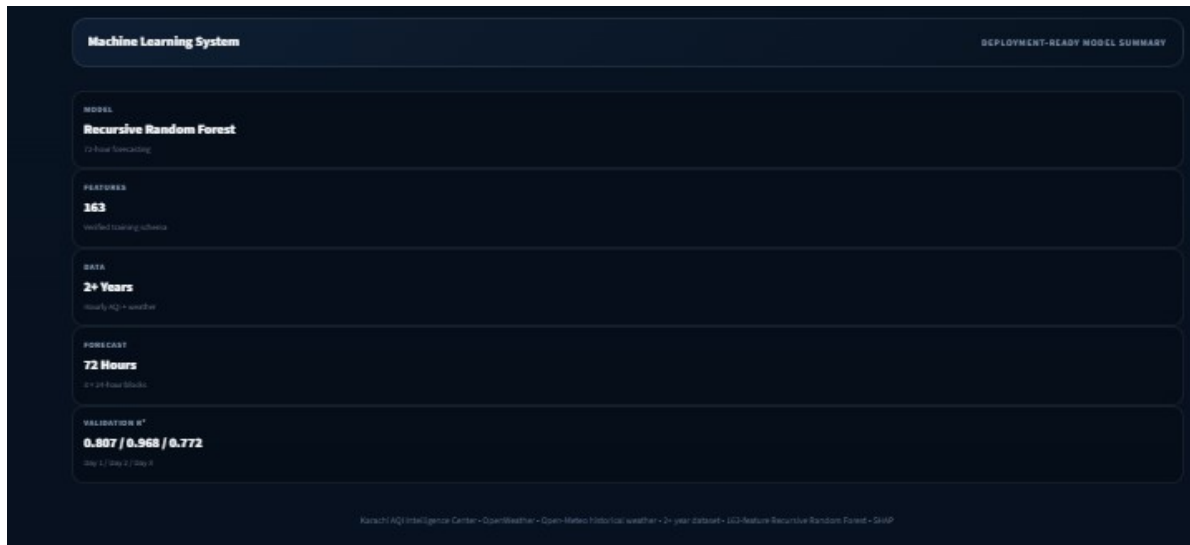


Figure 4. Dashboard system summary showing the final recursive model, 163-feature schema, 2+ years of data, and 72-hour forecast.

18. Hopsworks Feature Store and Model Registry

Hopsworks is integrated as the project's feature-store/model-management platform. The existing training infrastructure includes Feature Store access and a Model Registry registration function. The final recursive pipeline was then extended to register its own final model rather than leaving the strongest model only as a local artifact.

18.1 Final registered model

Property	Value
Registered name	karachi_aqi_recursive_random_forest_72h
Registry	Hopsworks Model Registry
Version verified	1
Feature count	163
Target scale	0–500 AQI
Forecast horizon	72 hours
Artifact	recursive_random_forest_compressed.pkl

During registration, Hopsworks emitted a provenance warning that the model schema could not be inferred automatically without both a Feature View and a training dataset version. Registration itself succeeded and the model was uploaded. This warning is retained as a limitation rather than being presented as full end-to-end provenance metadata.

19. CI/CD Automation with GitHub Actions

The project uses GitHub Actions with two scheduled jobs. The feature pipeline runs hourly at minute 0 of each hour. The training pipeline runs daily at midnight UTC. The final daily job now executes `recursive_72h.py` rather than the earlier 98-feature training script.

Schedule	Job	Command	Purpose
Hourly: 0 * * * *	Hourly Feature Pipeline	python feature_pipeline.py	Collect current data and generate operational features
Daily: 0 0 * * *	Daily Training Pipeline	python recursive_72h.py	Train, evaluate, save, and

Schedule	Job	Command	Purpose
			register final recursive model
Manual	workflow_dispatch	Configured in workflow	Allow an operator to trigger the pipeline on demand

The daily recursive script now performs training, 72-hour validation, compressed artifact creation, and Hopsworks Model Registry registration. This closes the major gap between the experimental final model and the automated project workflow described in the brief. [1]

20. Flask/API Component

The repository also contains a Flask-based API component. The public user-facing experience is currently Streamlit, while the Flask component provides an HTTP/API-oriented integration point for deployment scenarios that require a programmatic interface. Because the Streamlit dashboard is the verified public deployment, the report does not claim that Flask is the primary public UI.

21. Deployment and Public Accessibility

The final Streamlit application was deployed to Streamlit Community Cloud from the public GitHub repository. The platform supports repository-based deployment, custom app subdomains, secrets, and automatic propagation of committed code changes. [4]

Live application: <https://karachi-aqi-intelligence.streamlit.app/>

GitHub repository: <https://github.com/maryam-channa/karachi-aqi-predictor>

The deployed Streamlit app uses Python 3.11 and a requirements.txt that includes SHAP and its runtime dependencies. The deployment issue encountered during setup was a missing SHAP package declaration; adding shap==0.51.0 and verifying pip check resolved that dependency problem before the final deployment.

22. Security and Secret Management

API keys and Hopsworks credentials are not committed to the repository. For Streamlit Community Cloud, secrets are supplied through the platform's encrypted Secrets configuration rather than stored in source code. This is consistent with Streamlit's documented deployment model for secrets. [4]

- OPENWEATHER_API_KEY is kept outside Git history.
- HOPSWORKS_HOST, HOPSWORKS_PROJECT and HOPSWORKS_API_KEY are provided as runtime secrets where required.
- The repository contains model artifacts and data, but not the secret values themselves.

23. Limitations and Methodological Caveats

- The source pollution dataset contains 18 temporal gaps totaling 552 missing hours. The project preserves these gaps and avoids constructing features across discontinuities.
- Historical weather comes from Open-Meteo, while operational pollution/current inputs come from OpenWeather. Source differences may introduce distribution or spatial-resolution differences.
- The final recursive historical validation uses known future pollutant/weather fields from the historical window while replacing only AQI with recursive predictions. This is a controlled historical

evaluation and is not identical to a blind real-world 72-hour forecast in which future meteorological and pollutant inputs are themselves uncertain.

- The public dashboard uses packaged/local final model artifacts for inference together with operational inputs; it should not be described as performing live inference directly from a Hopsworks Feature View unless that integration is later implemented.
- The Hopsworks registry warning indicates that automatic model provenance/schema inference was incomplete because a Feature View and training dataset version were not supplied to the registration call.
- The 0–500 AQI target is the project's implemented AQI representation. The report does not claim exact identity with any particular government AQI methodology.
- Validation R^2 values are historical model-validation metrics. They are not “live R^2 ” values and do not guarantee future real-world accuracy.
- Recursive forecasting can accumulate errors over time. The Day 3 result remaining above the threshold is encouraging, but it does not remove this structural risk.

24. Future Improvements

- Use dedicated 72-hour meteorological forecasts instead of relying on controlled historical inputs for evaluation and static/held-forward weather assumptions in live testing.
- Connect the final recursive inference path directly to Hopsworks Feature Store / Model Registry where practical, reducing dependence on packaged local artifacts.
- Add explicit Feature View and training-dataset provenance to Hopsworks model registration.
- Add uncertainty estimates or prediction intervals using a statistically justified method rather than displaying unsupported confidence numbers.
- Experiment with LSTM/GRU/Temporal Fusion or other sequence models using the same leakage-controlled evaluation protocol.
- Introduce automated drift monitoring and model-performance monitoring after deployment.
- Expand spatial coverage beyond a single Karachi coordinate and evaluate city/zone-level forecasting.
- Add stronger automated data-quality tests around API outages, stale timestamps, and abrupt distribution shifts.

25. Conclusion

The Karachi AQI Predictor evolved from a basic one-step AQI prediction workflow into a complete, publicly accessible 72-hour forecasting system. The final implementation includes a two-plus-year historical dataset, a 0–500 AQI target, historical weather integration, 163 engineered features, a recursive Random Forest forecasting engine, SHAP explanations, Streamlit analytics, Hopsworks Model Registry integration, and GitHub Actions automation.

The strongest evidence of technical completion is the final held-out recursive validation: Day 1 $R^2 = 0.807421$, Day 2 $R^2 = 0.967712$, and Day 3 $R^2 = 0.772471$. Every forecast block exceeded the required $R^2 > 0.70$ threshold. The model was also registered in Hopsworks and the final dashboard is publicly accessible through Streamlit Community Cloud.

The project therefore meets the core deliverables defined in the brief: an end-to-end AQI prediction system, an automated pipeline, an interactive real-time/forecast dashboard, and a detailed technical

report. The remaining limitations are documented transparently and provide a clear roadmap for a more robust production-grade forecasting system.

26. References

- [1] Project Brief, “Pearls AQI Predictor / AQI Prediction Service,” uploaded project specification, 8 pages. Sections: Feature pipeline, Backfill historical data, Training pipeline, Automate pipeline runs, Web app, Guidelines and Final Submissions.
- [2] OpenWeather, “Air Pollution API.” <https://openweathermap.org/api/air-pollution>
- [3] Open-Meteo, “Historical Weather API.” <https://open-meteo.com/en/docs/historical-weather-api>
- [4] Streamlit, “Deploy your app on Community Cloud” and “Share your app.” <https://docs.streamlit.io/deploy/streamlit-community-cloud/deploy-your-app/deploy>
- [5] Streamlit, “Community Cloud.” <https://docs.streamlit.io/deploy/streamlit-community-cloud>
- [6] Project source repository: <https://github.com/maryam-channa/karachi-aqi-predictor>
- [7] Public deployed application: <https://karachi-aqi-intelligence.streamlit.app/>

27. Appendix: Final Repository and Evidence

A. Final repository structure

- app1.py
- api.py
- recursive_72h.py
- live_recursive_test.py
- backfill_2years.py
- backfill_weather_2years.py
- calculate_aqi_0_500.py
- merge_2years_data.py
- data/historical_aqi_2years_raw.csv
- data/historical_aqi_2years_aqi.csv
- data/historical_weather_2years.csv
- data/karachi_aqi_weather_2years.csv
- models/recursive_72h/imputer.pkl
- models/recursive_72h/metadata.pkl
- models/recursive_72h/recursive_random_forest_compressed.pkl
- training/train_models_v1.py
- deployment/predictor.py
- eda.py
- eda_results/
- .github/workflows/main.yml
- requirements.txt
- requirements-tensorflow.txt
- README.md

B. Final Git history highlights

Commit	Purpose
de41bfa	Finalize 3-day AQI forecasting system
f0ca63c	Improve project documentation and dependencies

Commit	Purpose
8dd2911	Automate final 72-hour model training
20bc1b4	Finalize dashboard and recursive model artifacts
6586828	Polish dashboard validation section

C. Final evidence checklist

Evidence Item	Verified Result
Python compilation	Key files compile successfully
pip check	No broken requirements found
SHAP import	SHAP 0.51.0 imports successfully
Core imports	Streamlit, pandas, numpy, sklearn, Hopsworks, SHAP, Flask, Plotly all import successfully
Recursive validation	Day 1/2/3 R ² all PASS
Hopsworks registry	Registration PASSED; version 1 created
Git push	HEAD synchronized with origin/main; working tree clean
Public dashboard	Streamlit Community Cloud deployment verified

D. Final model metrics (full precision)

Metric	Value
One-step RMSE	1.585224
One-step MAE	0.580212
One-step R ²	0.993270
Day 1 RMSE	0.505405
Day 1 MAE	0.431990
Day 1 R ²	0.807421
Day 2 RMSE	0.424322
Day 2 MAE	0.362010
Day 2 R ²	0.967712
Day 3 RMSE	0.599883
Day 3 MAE	0.496560
Day 3 R ²	0.772471

End of Report